Australian National University

Venue _____

Student Number |__|__|__|__|__|__|__|__|

## School of Computing

# EXAMINATION

Semester 1 -End of Semester, 2024

## COMP2300_COMP6300_ENGN2219 Semester 1

## Computer Organization and Program Execution

*This paper is for ANU students.*

Examination Duration: 180 minutes

Reading Time: 15 minutes

**Exam Conditions:**

No materials permitted

**Materials Permitted In The Exam Venue:**

**(No electronic aids are permitted e.g. laptops, phones)**

No materials permitted

**Materials To Be Supplied To Students:**

1 x 20 pages

**Instructions To Students:**

**The exam is marked out of 100.** Each question's total marks and the breakdown of marks are indicated before each question. You should spend more time on questions with more weight.

| | | | |
|---|---|---|---|
| **Q1** | **Multiple Choice** | 20 | |
| **Q2** | **Digital Logic** | 10 | |
| **Q3** | **Timing Issues** | 10 | |
| **Q4** | **Finite State Machine** | 14 | |
| **Q5** | **ARM Assembly** | 10 | |
| **Q6** | **Input/Output** | 6 | |
| **Q7** | **Microarchitecture** | 30 | |
| **Aggregate Marks** | | 100 | |

- Write your student number and answers clearly in good handwriting.

- **An ARM cheat sheet is provided at the end of the exam sheet (after page # 21).**

- Any formula sheet is neither deemed necessary nor provided for this closed-book exam.

- All the arithmetic required to answer questions is simple as long as you are comfortable with the binary number system. The use of a calculator is not deemed necessary for this exam. Using a calculator or a mobile aid is strictly forbidden.

- This exam has seven main questions and several sub-questions. The first question consists of 20 multiple-choice and true/false questions on a variety of topics. The subject of each remaining question is indicated.

- Indicate your answers clearly, and do not make multiple selections. Think carefully before selecting your answer.

- Do not attach extra sheets. Write your answers in the space provided in the question sheet.

- We provide an extra sheet after the final exam question. If you use that extra sheet to provide answers to any questions, clearly mention page number 21 so we can mark your answers.

- In the upcoming question, there is only one correct answer. Circle only one answer.

- Clearly read the marking scheme of the first question. There is negative marking in the first question only. Avoid random guesses as it might negatively affect your score.

**Multiple choice and true/false questions [20, 1 for correct, -1 for wrong, 0 for no selection]**

**Q1a.** How many bits are needed to represent –128 with sign/magnitude numbers?

- 7
- 8
- 9
- 10
- None of the above

**Q1b.** If one of the inputs of a NAND gate is connected to logic level 1 (TRUE) forever, the resulting circuit behaves like:

- OR gate
- NOR gate
- NOT gate
- Short circuit wire
- Tri-state buffer

**Q1c.** Under De Morgan's Theorem, which of the following circuits are equivalent:

- An AND gate with inverted output *is equivalent to* an AND gate with inverted inputs
- An AND gate with inverted output *is equivalent to* a NOR gate with inverted output
- An AND gate with inverted output *is equivalent to* an OR gate with inverted inputs
- An OR gate with inverted output *is equivalent to* an OR gate with inverted inputs
- A NOR gate with inverted output *is equivalent to* an AND gate with inverted inputs
- None of the above

**Q1d.** A 2:4 decoder can be used to implement an XNOR gate by connecting which ones of its one-hot outputs to the inputs of an OR gate:

- 00 and 01
- 00 and 11
- 01 and 10
- 10 and 11
- 00 and 10

**Q1e.** Two NAND gates are cross-coupled to produce outputs X and Y. Their remaining terminals are connected to inputs A and B. When A = 1 and B = 1, which one of the following statements is correct?

- X is 1, and Y is 0
- X is 0, and Y is 1
- X and Y remember their old values
- X and Y oscillate between 1 and 0
- X and Y are not complements of each other

**Q1f.** What happens to circuit complexity (ignore wires) when the addressability of memory is increased from 8 to 16 bits without changing the number of locations (rows)? (Decoding logic generates one-hot word lines. Multiplexing logic selects one word out of many rows.)

- An increase in memory cell count, but the complexity of decoding logic reduces
- An increase in memory cell count and the complexity of decoding logic
- An increase in memory cell count, but the complexity of multiplexing logic reduces
- An increase in memory cell count and the complexity of multiplexing logic
- An increase in memory cell count and the complexity of decoding and multiplexing logic

**Q1g.** Whether or not all steps necessary to process an instruction are performed in one cycle or more is determined by:

- ISA
- Microarchitecture
- None of the above

**Q1h.** The limit to how far branch instructions can jump is determined by:

- ISA
- Microarchitecture
- None of the above

**Q1i.** Deciding which registers are preserved or not preserved by functions is determined by:

- ISA
- Microarchitecture
- None of the above

**Q1j.** What is the execution time (in seconds) of a program that executes 2 billion instructions on a CPU with a clock frequency of 2 GHz and an average IPC of 0.25?

- 0.5
- 1
- 2
- 4
- 8

**Q1k.** Two CPUs, P and E, in the market have a power rating of 120 Watts and 80 Watts, respectively. A program X executes twice as fast on P than on E. If preserving battery life is the goal, then the best course of action is:

- Execute X on P
- Execute X on E
- Execute X on either P or E
- X cannot execute twice as fast on P
- None of the above

**Q1l.** A C program is compiled into ARM (A) and MIPS (M) assemblies. W and X are ARM CPUs with different critical paths through their logic. Y and Z are MIPS CPUs with similar frequency. The assembly program A is run on W and X, and M is run on Y and Z. In which of the following scenarios is it okay to use instructions per cycle or IPC as a metric for comparing the performance of two CPUs?

- A on W *versus* M on Y
- A on X *versus* M on Z
- A on W *versus* A on X
- M on Y *versus* M on Z
- A on X *versus* M on Y

**Q1m.** A compiler translates high-level code into assembly. Which of the following metrics is NOT impacted by the compiler?

- Energy consumption
- Cycles per instruction
- Seconds per cycle
- Stack frame size
- Instruction mix

**Q1n.** Register R0 contains the index of an element in a C array. The base address of the array in memory is 0x0. Each array element is 32 bytes in size. Which of the following instructions writes the memory address of the array element in register `R1`?

- `LSL R1, R0, #2`
- `LSL R1, R0, #3`
- `LSL R1, R0, #4`
- `LSL R1, R0, #5`
- `LSL R1, R0, #6`

**Q1o.** Which of the following is an example of shifting the “engineering” burden from hardware to software?

- ISA with complex instructions
- ISA with simple instructions
- Dynamic scheduling
- Microprogrammed control unit
- Complex addressing modes

**Q1p.** Executing interrupt handers most closely resembles which of the following:

- Early branch resolution in a pipelined CPU
- Updating `PC` to `PC` + 4
- Function execution
- Conditional branch execution
- Addressing a memory-mapped IO device

**Q1q.** Which of the following instructions is an example of a true dependency?

- `ADD R0, R1, R2` *followed by* `ADD R3, R4, R5`
- `LDR R0, [R1]` *followed by* `STR R0, [R1]`
- `ADD R0, R1, R2` *followed by* `SUB R0, R4, #16`
- `ADD R0, R1, R2` *followed by* `ADD R1, R1, R7`
- None of the above

**Q1r.** The accuracy of branch prediction is NOT determined by:

- Strength of the branch predictor
- Loop termination conditions
- Format of the branch instruction
- Size of the branch history table
- Frequency of branch instructions

**Q1s.** Consider a load (L) that incurs a cache miss, followed by an instruction (M) dependent on L, followed by an instruction (N) that does not depend on L or M. A dynamically scheduled CPU will:

- Stall L and M and N
- Stall L, but not M and N
- Stall L and N, but not M
- Stall L and M, but not N
- None of the above

**Q1t.** In general, instruction-level parallelism is NOT inhibited by:

- High frequency of memory instructions
- High frequency of branch instructions that often change direction
- High frequency of young and dependent ALU instructions
- High frequency of young and independent ALU instructions
- High frequency of `PC`-changing instructions

**Digital Logic Fundamentals [2 + 4 + 2,  1 + 1]**

**Q2a.** One day at work, you are tasked with designing a combinational circuit with a three-bit input, {A, B, C} (A is the most significant bit, and C is the least significant bit), and one output, Y. The value of Y is 1 when the 3-bit input contains two or more ones, with one exception. The exceptional case occurs when A and C are 1 and B is 0. In that case, the output Y is 0.

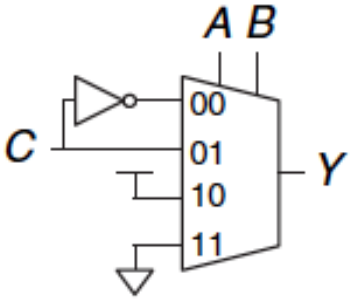
Answer the next three questions. You can also use the white space on the next page if required.

A. Draw the truth table for the combinational circuit.

B. Express the output Y as the simplest sum of product representation. Show your work step-by-step. Your work should first show the expanded Boolean equations translated from the truth table.

C. Using basic logic gates, draw a schematic of Y as a function of A, B, and C.

**Q2b.** The following circuit shows an example of implementing logic using multiplexers. The “10” input of the multiplexer is connected to logic level 1 or TRUE, and the input “11” is connected to logic level Ground or 0. Determine the output Y for the following two cases. Write your answer next to the options below. You can assume that A is the most significant bit.

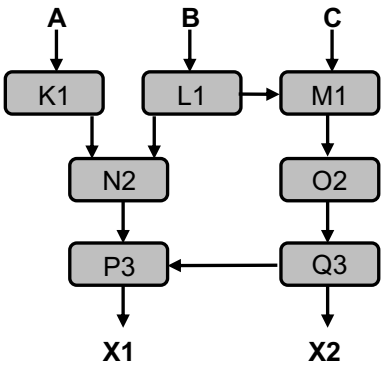



A. $A = 0$, $B = 0$, $C = 0$, $Y =$ ____

B. $A = 1$, $B = 1$, $C = 1$, $Y =$ ____

**Timing in Digital Circuits [2 + 2 + 2, 4]**

**Q3a.** Consider the high-level schematic of a general-purpose combinational hardware circuit shown that computes two outputs from three inputs. Circle the correct answer.




A. The correct statement about the relationship of X1 and X2 to the three inputs is:

   a. X1 and X2 both depend on all three inputs

   b. X1 and X2 both depend on only two of the three inputs

   c. X1 depends on all three inputs, A, B, and C, but X2 depends on only two of the three inputs

   d. X2 depends on all three inputs, A, B, and C, but X1 depends on only two of the three inputs

   e. None of the above is correct

B. If all components have a delay of 4 units, then the critical path of X1 consists of:

   a. K1, N2, P3

   b. L1, M1, O2, Q3, P3

   c. L1, N2, P3

   d. L1, N2, Q3, P3

   e. M1, O2, Q3, P3

C. If K1 has a delay of 16 units, and all other components have a delay of 4 units, then the critical path of X1 consists of:

   a. K1, N2, P3

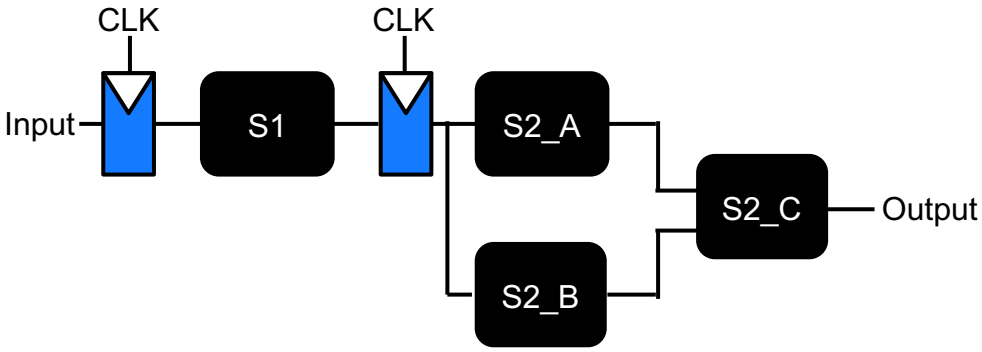
   b. L1, M1, O2, Q3, P3

   c. L1, N2, P3

   d. L1, N2, Q3, P3

   e. M1, O2, Q3, P3

**Q3b.** The following sequential circuit processes its input in a staged manner. The combinational elements are labeled S1, S2_A, S2_B, and S2_C. The registers receive the clock (CLK) input. The circuit shown is a purpose-built accelerator for solving a high-performance computational problem. Note that the second stage is divided into several combinational circuits labeled A, B, and C. Assume a new input is available to S1 at the rising edge of a clock cycle. The circuitry (not shown) receiving the output signal, Output, stores it in a register for one full clock cycle. The timing specification for the combinational and sequential elements of the circuit is given below. What is the time (in nanoseconds) that it takes to process 10 inputs?




Timing specification

- The propagation delay of S1 is equal to 4.4 nanoseconds.
- The propagation delay of Stage 2 (A) is equal to 4.0 nanoseconds.
- The propagation delay of Stage 2 (B) is equal to 3.8 nanoseconds.
- The propagation delay of Stage 2 (C) is equal to 0.5 nanoseconds.
- The clock-to-Q propagation delay ($t_{pcq}$) is equal to 0.3 nanoseconds.
- The setup time ($t_{setup}$) is equal to 0.2 nanoseconds.
- The hold time ($t_{hold}$) is zero.

Write your answer below.

**Finite State Machines [5 + 3 + 4 + 2]**

**Q4.** Design a finite state machine (FSM) with one input, A, and one output, X. The output X should be 1 if A has been 1 for three consecutive clock cycles. Otherwise, X should be 0.

A. Design the Moore state transition diagram for the above specification.

B. Build the encoded state transition tables.

C. Specify and simplify the next state and output equations.

D. Draw the high-level schematic of the FSM. You do not need to show the gate-level logic, but you should show the state register, next stage component, feedback loop, and output component.

Write your answers on this page and the next page.

**ARM Assembly and ISA [10]**

**Q5.** Translate the following C function (starting at the label `func`) into 32-bit ARM assembly code. The parameters of the function are provided in `R0` and `R1`. The result should be returned in `R0`.

Your code should preserve registers using the ARM calling convention, if necessary.

```
int func(int n, int m) {
      int x = 0;
      int i = 0;
      int j = 0;

      while (i < n) {
            x = x + 1;
            if (j == m) {
                  x = x + m;
                  j = 0;
            }
            i = i + 1;
            j = j + 1;
     }
    return x;

}
```

Write your code in the space provided on this page and the next page (if needed).

**Input/Output [3, 3]**

**Q6a.** It has been observed that at a bank, during peak hours, the server processing client requests spends 60% of the time where the CPU is 100% busy and 40% of the time waiting for a response from disk. If the CPU is upgraded to make it 500% faster than the existing CPU, what total speedup can the banking service expect?

**Q6b.** An embedded computer system uses memory-mapped I/O devices. A specific I/O device is assigned the memory address 0x30000000. An I/O register is associated with the memory address, and its output connects to the input pins of the I/O device. If the $8^{th}$ bit of the I/O register is set, the I/O device goes into standby mode. Show the 32-bit ARM assembly code for sending the I/O device into standby mode.

**Microarchitecture [5 + 5 + 5, 1 + 2, 6 + 2 + 2 + 2]**

**Q7a.** This question is about executing the following instruction sequence on a *scalar* in-order pipelined CPU. First, study the instruction sequence below. Then, answer the questions.

```
i1:   MOV R0, #10
i2:   MOV R1, #0
i3:   MOV R2, #256
i4:   MOV R3, #0
i5:   MOV R4, #0
     LOOP:
i6:   CMP R0, R1
i7:   BEQ DONE
i8:   LDR R3, [R2], 4
i9:   LDR R4, [R3, #0]
i10:  ADD R1, R1, #1
i11:  ADD R4, R4, R1
i12:  B LOOP
     DONE:
```

A. Consider the ARM five-stage pipelined CPU with fetch, decode, execute, memory, and writeback stages. There is no forwarding and hazard detection in the hardware. Your task is to insert NOPs in the assembly code to execute the instructions correctly on the pipelined CPU. Rewrite the code above by inserting NOPs. *Your answer is only valid if you use as few NOPS as possible.* You do not need to write complete instructions. You can use instruction labels, i1, i2, etc.

B. Now consider the ARM five-stage pipelined CPU with forwarding and hazard detection in hardware. In the case of conditional branches, this CPU predicts branches are not taken. You can make the following assumptions: (1) Branches resolve and update the PC at the end of the writeback stage, (2) Memory accesses take one cycle.

Identify all pairs of instructions where forwarding helps avoid stalls altogether. Then, identify all pairs of instructions that lead to one or more stall cycles despite hazard detection.

**C.** How many cycles does the program take executing on the CPUs in (A) and (B)?

**Q7b.** Consider the following ARM assembly code. Answer the following questions.

```
i1: LDR R1, [R0, #16]
i2: ADD R3, R1, #32
i3: LDR R2, [R0, #32]
i4: ADD R4, R2, #32
i5: SUB R1, R6, #4
i6: SUB R2, R1, R7
```

A. Suppose the above straight-line instruction sequence is executed on a computer with an in-order scalar pipelined CPU that stalls on a read-after-write hazard. Suppose the memory system of this computer allows two memory accesses to be performed concurrently. Reorder the above instruction sequence by hand to take advantage of this memory-level parallelism feature. Write your new code below.

B. Consider the above (original) instruction sequence again. Suppose you are provided two hypothetical registers, S and T. Using S and T, rename the registers in the above instruction sequence to eliminate all false dependences. Your resulting (renamed) instruction sequence must still respect all true dependencies. Write your new code below.

**Q7c.** This question requires you to complete the pipeline diagram and show the contents of different structures of a scalar out-of-order processor that uses a CDC 6600-style scoreboard for resolving read-after-write hazards. CDC 6600 uses dynamic scheduling but no speculation. Use the following assumptions.

- A WAR or WAW hazard stalls instruction fetch in the RR stage.
- The CPU has an 8-entry architectural register file. The contents of the register file are shown below.
- All scoreboard entries are 1 (register is read to read) before the execution of the first instruction. A busy register, i.e., value is being computed, and dependent instruction must wait, is indicated by {v = 0} in the scoreboard. The scoreboard is shown below.
- The issue queue is empty. All slots are available (v = 0). An unavailable slot is indicated by the v bit set to 1. It stores the destination tag (dst tag), source tags or values (rs1/rs2 tag/value), and bits to indicate if source operands are ready (rs1/rs2 rdy). The issue queue is shown below.
- The pipeline is fully flushed before the execution of the first instruction.
- Any memory access results in a cache miss.
- Memory accesses are completed in three steps: (1) address generation, (2) data cache access, and (3) main memory access. The request is sent to memory during the next cycle if data is not present in the cache.
- It takes four cycles for the memory to respond. Therefore, an access that does not find the requested word in the data cache spends 1 cycle computing the address, 1 cycle looking up in the data cache, and four cycles waiting for memory (6 cycles total in the EX-stage).
- The value of the data word stored at memory location 32 is 0.

Nest, consider the four instructions below to answer the questions that follow.

```
i1: ADD R0, R2, R3
i2: ADD R7, R0, R4
i3: LDR R1, [R5, #0]
i4: ADD R6, R1, #4
```

The initial contents of the scoreboard, the register file, and the issue queue are shown below.

| | v |
|---|---|
| R0 | 1 |
| R1 | 1 |
| R2 | 1 |
| R3 | 1 |
| R4 | 1 |
| R5 | 1 |
| R6 | 1 |
| R7 | 1 |

| | value |
|---|---|
| R0 | #16 |
| R1 | #16 |
| R2 | #16 |
| R3 | #12 |
| R4 | #24 |
| R5 | #32 |
| R6 | #12 |
| R7 | #16 |

| v | dst tag | rs1 rdy | rs1 tag/value | rs2 rdy | rs2 tag/value |
|---|---|---|---|---|---|
| 0 | | | | | |
| 0 | | | | | |
| 0 | | | | | |

A. Fill in the cycle-by-cycle pipeline diagram using labels for pipeline stages below.

Fetch = FE, Decode = DE, Register Read = RR, Dispatch = DI, Issue = IS, Execute = EX, Address Generation = E@, Cache Miss/Memory access = EX$, Writeback = WB

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| i1: ADD R0, R2, R3 | | | | | | | | | | | | | | | | | |
| i2: ADD R7, R0, R4 | | | | | | | | | | | | | | | | | |
| i3: LDR R1, [R5, #0] | | | | | | | | | | | | | | | | | |
| i4: ADD R6, R1, #4 | | | | | | | | | | | | | | | | | |

B. Provide the contents of the issue queue at the end of cycle # 9. You can pick any free slot in the issue queue for any instruction. Use the following format to answer this question using text.

Cycle # X: { v=1, dst tag = `R0`, rs1 rdy = 0, rs1 tag = `R0`, rs2 rdy = 0, rs2 val = 0 }

C. Provide the contents of the scoreboard at the end of cycle # 6. You can show the scoreboard contents by writing a string of 8 binary digits, e.g., 00001111.

D. Provide the contents of the register file at the end of cycle # 9. Use the format below.

{`R0` = 10, `R1` = 10, `R2` = 13, and so on for the remaining registers}

**END OF EXAMINATION**